



Programa de Residência em Microeletrônica
MCTI / Softex / IRede

ATIVIDADE AVALIATIVA
Controlador de Vending Machine
em SystemVerilog

Trilha: RTL Design — Módulo: Projeto de Controlador Digital
Trabalho individual ou em duplas

⚠ Leia o documento completo antes de escrever qualquer linha de código.

1. Objetivo

Nesta atividade você irá projetar, codificar, simular e sintetizar o controlador digital de uma vending machine (máquina de venda automática) utilizando SystemVerilog, Synopsys VCS e Design Compiler. O projeto integra os três elementos fundamentais de um sistema digital síncrono: máquina de estados finitos (FSM), memória de dados e lógica combinacional de caminho de dados.

Ao final você será capaz de:

- Projetar e codificar uma FSM de Moore com múltiplos estados em SystemVerilog
- Integrar memória síncrona de dados a uma FSM de controle
- Separar claramente caminho de dados (datapath) e unidade de controle
- Compilar, simular e depurar com Synopsys VCS e Verdi
- Escrever um testbench self-checking com múltiplos cenários de teste
- Sintetizar o design com Design Compiler e analisar os relatórios gerados

2. Descrição do Sistema

O controlador gerencia uma *vending machine* com 4 itens (café, água, suco e snack). O usuário insere moedas, seleciona um item, confirma a compra e recebe o produto e o troco. O controlador não possui programa armazenado — toda a lógica de sequenciamento é implementada pela FSM.

Características principais:

- Design totalmente síncrono com clock e reset globais
- Memória de dados interna com preços e estoques dos 4 itens
- Acumulador de crédito (registrador síncrono)
- Comparador e subtrator combinacionais no caminho de dados
- FSM de Moore com 6 estados e saídas registradas

3. Diagrama de Blocos

- Sua primeira tarefa é apresentar um diagrama de blocos desse sistema, contemplando:

- **Entradas:**

- ◆ coin_in[1:0]
- ◆ sel_item[1:0]
- ◆ confirm - 1bit
- ◆ cancel - 1bit
- ◆ clk (Clock global)
- ◆ rst (reset síncrono)

- **Registrador de crédito:**

- ◆ credit[7:0]
- ◆ O registrador de crédito é um registrador síncrono de 8 bits que acumula o valor total das moedas inseridas pelo usuário ao longo do tempo. Cada vez que o usuário insere uma moeda e a FSM está no estado COLLECT, o sinal `credit_load` é ativado e o registrador soma o valor da moeda ao crédito atual:
- ◆ $credit \leftarrow credit + coin_value$
- ◆ onde coin_value é derivado de coin_in:
 - 00 → R\$0,00 (nenhuma moeda)
 - 01 → R\$0,25

- 10 → R\$0,50
- 11 → R\$1,00
- ◆ Use representação em ponto fixo de maneira que internamente:
 - 0,25 é representado por 25
 - 0,50 é representado por 50
 - 1,00 é representado por 100
- ◆ Com 8 bits sem sinal você consegue representar de 0 a 255 centavos — o suficiente para cobrir todos os itens da máquina (o mais caro custa 100 centavos) e acumular até R\$2,55 de crédito.
- **Comparador**
 - ◆ O comparador verifica duas condições ao mesmo tempo para decidir se a venda pode ser realizada. Ele produz um único sinal combinacional `can_sell` que só vai para 1 se ambas forem verdadeiras:
 - `can_sell = (credit >= price) AND (stock > 0)`
 - ◆ **Condição 1 — crédito suficiente:** Compara o valor acumulado no registrador de crédito com o preço do item selecionado, lido da memória. Como ambos estão em centavos (inteiros), é uma simples comparação de magnitude entre dois números de 8 bits — o hardware implementa isso com um subtrator e verifica o sinal do resultado.
 - ◆ **Condição 2 — item disponível:** Verifica se o estoque do item selecionado é maior que zero. Basta checar se os 8 bits de `stock` são todos zero ou não.
 - ◆ **Por que é combinacional e não registrado:** O comparador não tem clock — ele calcula a saída instantaneamente a partir das entradas. A FSM lê a memória no estado CHECK, e no ciclo seguinte `price` e `stock` já estão disponíveis nos fios. O comparador avalia `can_sell` nesse mesmo ciclo, e a FSM usa esse valor para decidir se vai para DISPENSE ou ERROR.
 - ◆ **O que ele não faz:** Ele não toma nenhuma decisão — apenas calcula um resultado booleano. Quem decide o que fazer com esse resultado é a FSM, no estado CHECK.
- **Subtrator**
 - ◆ Calcula o troco em centavos
 - `change = credit - price`
 - ◆ **Quando o resultado é usado?**
 - O subtrator é combinacional. Ele está calculando `credit - price` o tempo todo, mas o resultado só importa quando a FSM entra no estado CHANGE. Nesse momento a FSM ativa o sinal que registra `change` na saída `change_out`, e zera o crédito.
 - ◆ **O troco pode ser negativo?**
 - Não, porque o comparador já garantiu antes que `credit >= price` antes de a FSM ir para DISPENSE. Quando chega no estado CHANGE, a subtração é sempre não-negativa. Essa é exatamente a divisão de responsabilidades entre os dois módulos: o comparador garante que a conta é válida, o subtrator só executa a conta.
 - ◆ **E se o crédito for exato?**
 - `change = 0` — `change_out` fica em zero, nenhum troco é devolvido. A FSM passa por CHANGE normalmente e volta para IDLE.
- **Unidade de controle:**
 - ◆ A FSM tem 6 estados, organizados em sequência linear com dois desvios possíveis:
 - ◆ **IDLE:** Estado inicial e de repouso. A máquina está esperando. Nenhuma saída ativa. Permanece aqui até detectar `coin_in ≠ 00`.
 - ◆ **COLLECT:** Acumula moedas. A cada ciclo que `coin_in ≠ 00`, ativa `credit_load` e soma o valor ao registrador de crédito. O usuário pode inserir várias moedas consecutivas antes de confirmar. Quando `confirm = 1`, avança para CHECK.

- ◆ **CHECK:** Ativa `mem_read` para buscar `price` e `stock` do item selecionado na memória. No ciclo seguinte o comparador já tem os valores e produz `can_sell`. A FSM então decide:
 - `can_sell = 1` → vai para DISPENSE
 - `can_sell = 0` → vai para ERROR
- ◆ **DISPENSE:** Ativa `dispense` por exatamente 1 ciclo (pulso) — o mecanismo físico de liberação do item detecta essa borda. Também ativa `mem_write` para decrementar o estoque na memória (`stock ← stock - 1`). Avança imediatamente para CHANGE.
- ◆ **CHANGE:** Calcula e registra o troco (`change_out = credit - price`) e zera o crédito (`credit_load = 1, credit ← 0`). Depois retorna ao IDLE.
- ◆ **ERROR:** Ativado quando `can_sell = 0`, o que indica crédito insuficiente ou sem estoque. Mantém `error = 1` e aguarda `cancel` para voltar ao IDLE. O crédito é devolvido ao usuário via `change_out`.
- ◆ O fluxo normal completo é sempre:
 - IDLE → COLLECT → CHECK → DISPENSE → CHANGE → IDLE
- ◆ E o desvio de erro:
 - CHECK → ERROR → IDLE
- ◆ `cancel` em qualquer estado retorna imediatamente ao IDLE e zera o crédito.
- Saídas:
 - ◆ Combinacionais:
 - `dispense`
 - `error`
 - ◆ Registradas
 - `change_out`
 - `display`
 - `state_out`

4. Interface do Módulo Top-Level

4.1 Entradas

Sinal	Largura	Direção	Descrição
<code>coin_in</code>	2 bits	entrada	Valor da moeda inserida: 00=nenhuma, 01=R\$0,25, 10=R\$0,50, 11=R\$1,00
<code>sel_item</code>	2 bits	entrada	Seleção do item desejado (0..3). Também é o endereço da memória
<code>confirm</code>	1 bit	entrada	Confirmação da compra pelo usuário (pulso síncrono)
<code>cancel</code>	1 bit	entrada	Cancelamento: devolve crédito e retorna ao IDLE
<code>clk</code>	1 bit	entrada	Clock global síncrono
<code>rst</code>	1 bit	entrada	Reset síncrono ativo em nível alto

4.2 Saídas

Sinal	Largura	Descrição
<code>dispense</code>	1 bit	Pulso de 1 ciclo: sinaliza que o item deve ser liberado
<code>change_out</code>	8 bits	Valor do troco em centavos (<code>credit - price</code>). Válido no estado CHANGE
<code>error</code>	1 bit	Ativo quando sem estoque ou crédito insuficiente

Sinal	Largura	Descrição
display	8 bits	Crédito acumulado atual, para exibição em display externo
state_out	3 bits	Estado corrente da FSM (para depuração e testbench)

Observações importantes:

- coin_in é amostrado a cada borda de subida do clock. Um valor diferente de 00 indica inserção de moeda naquele ciclo.
- sel_item deve ser estável antes de confirm ser asserted. Ele também funciona como endereço direto na memória.
- dispense é um pulso de exatamente 1 ciclo de clock — o hardware externo de liberação deve detectar essa borda.
- cancel em qualquer estado retorna ao IDLE e zera o crédito.

5. Memória de Dados

A memória armazena preços e estoques dos 4 itens em 4 posições de 16 bits. Os 8 bits superiores contêm o preço em centavos; os 8 bits inferiores, o estoque disponível. O endereço é diretamente fornecido por sel_item [1:0].

Addr	Item	price [7:0]	stock [7:0]	Observação
0x0	Café	0x19 (25)	0x05	Preço R\$0,25 — 5 unidades iniciais
0x1	Água	0x32 (50)	0x05	Preço R\$0,50
0x2	Suco	0x4B (75)	0x03	Preço R\$0,75
0x3	Snack	0x64 (100)	0x02	Preço R\$1,00

Comportamento da memória:

- Leitura síncrona: quando mem_read=1, os campos price e stock ficam disponíveis no ciclo seguinte
- Escrita síncrona: quando mem_write=1, o campo stock é decrementado de 1 (stock ← stock - 1) no endereço sel_item
- A memória deve ser inicializada com os valores da tabela acima via initial begin
- Não há escrita no campo price — preços são fixos

6. Máquina de Estados — FSM de Moore

A unidade de controle é uma FSM de Moore: as saídas dependem apenas do estado atual, não das entradas. Implemente com dois blocos always: um always_ff para a transição de estados e um always_comb para a lógica de saídas.

Estado	Encoding	Condição de entrada	Saídas ativas / Ação
IDLE	3'b000	rst ou cancel (de qualquer estado)	Aguarda: nenhuma saída ativa

Estado	Encoding	Condição de entrada	Saídas ativas / Ação
COLLECT	3'b001	coin_in $\neq$ 00 em IDLE ou em COLLECT	credit_load=1; acumula coin_value no crédito
CHECK	3'b010	confirm=1 em COLLECT	mem_read=1; compara credit $\geq$ price e stock > 0
DISPENSE	3'b011	can_sell=1 em CHECK	dispense=1 (1 ciclo); mem_write=1 (decrementa stock)
CHANGE	3'b100	sempre após DISPENSE	change_out = credit - price; credit_load=1 (zera crédito)
ERROR	3'b101	can_sell=0 em CHECK	error=1; aguarda cancel para voltar ao IDLE

6.1 Diagrama de Transição de Estados

Desenhe o diagrama de estados de acordo com o que foi descrito na seção 2.

7. Caminho de Dados

7.1 Registrador de Crédito

Registrador síncrono de 8 bits que acumula o valor das moedas inseridas. Controlado pelo sinal credit_load da FSM.

- credit_load=1 e FSM em COLLECT: $\text{credit} \leftarrow \text{credit} + \text{coin_value}$
- credit_load=1 e FSM em CHANGE: $\text{credit} \leftarrow 0$ (zera após dispensar)
- cancel ou rst: $\text{credit} \leftarrow 0$

O valor coin_value é derivado de coin_in: 00→0, 01→25, 10→50, 11→100 (centavos).

7.2 Comparador

Lógica combinacional pura (always_comb ou assign):

```
assign can_sell = (credit >= price) && (stock > 8'b0);
```

7.3 Subtrator de Troco

Lógica combinacional pura, ativo no estado CHANGE:

```
assign change = credit - price; // resultado em centavos
```

O valor change_out é registrado na saída quando a FSM entra em CHANGE, e mantido até o próximo ciclo de operação.

8. Simulação com Synopsys VCS

Escreva um testbench simples (sem UVM) e um script de simulação de acordo com os exemplos passados dos roteiros anteriores

8.1 Cenários de Teste Obrigatórios

O *testbench* deve implementar e verificar automaticamente todos os cenários abaixo:

#	Cenário	Sequência de entradas	Verificação esperada
1	Compra bem-sucedida com troco	coin_in=11 (R\$1,00); sel_item=0 (café, R\$0,25); confirm=1	dispense=1; change_out=75; credit=0 ao final
2	Crédito insuficiente	coin_in=01 (R\$0,25); sel_item=3 (snack, R\$1,00); confirm=1	error=1; FSM vai para ERROR
3	Cancelamento	coin_in=11; coin_in=11; cancel=1	credit=0; FSM retorna a IDLE; change_out=200
4	Estoque zerado	Comprar café 5 vezes (estoque=5); tentar 6ª vez	Na 6ª: error=1 (stock=0)

8.2 Estrutura do Testbench

O testbench deve conter:

1. Geração de clock: `always #5 clk = ~clk;` (período 10 ns)
2. Reset inicial por 2 ciclos de clock
3. Tarefa `apply_coin(value)` que aplica uma moeda e aguarda 1 ciclo
4. Tarefa `buy_item(item, coins[])` que executa uma compra completa
5. Tarefa `check(expected, actual, label)` que reporta PASS/FAIL
6. Geração de waveform;

9. Síntese com Synopsys Design Compiler

Síntese e Exploração de Constraints com Synopsys Design Compiler

Com o design verificado em simulação, o próximo passo é a síntese lógica e a análise dos limites temporais do circuito. Esta etapa é composta de duas partes: a escrita do script de síntese e a exploração sistemática das constraints de timing.

Parte 1 - Script de síntese

Escreva um script Tcl (`synth.tcl`) para o Synopsys Design Compiler que realize as seguintes etapas, nesta ordem:

1. Configure `target_library` e `link_library` apontando para a biblioteca disponível no ambiente do laboratório.
2. Leia e analise todos os arquivos RTL com `analyze -format sverilog`.
3. Elabore o design com `elaborate`, definindo `vending_top` como módulo de topo, e execute `link`.
4. Leia o arquivo de constraints com `read_sdc vending.sdc`.

5. Execute `check_design` e salve o resultado em `reports/check_design.rpt`. Corrija todos os erros antes de prosseguir.
6. Execute a síntese com `compile_ultra -no_autoungroup`.
7. Gere e salve os seguintes relatórios: `report_area`, `report_timing`, `report_power` e `report_constraint -all_violators`.
8. Exporte a netlist sintetizada com `write -format verilog -hierarchy`.

Parte 2 — Arquivo de constraints

Escreva o arquivo `vending.sdc` definindo as seguintes constraints:

1. Crie um clock de nome `clk` com período inicial de 20 ns (50 MHz) usando `create_clock`.
2. Defina `set_clock_uncertainty` com valor de 0,5 ns para modelar jitter e skew.
3. Defina `set_input_delay` de 3 ns para todas as entradas em relação ao clock.
4. Defina `set_output_delay` de 3 ns para todas as saídas em relação ao clock.
5. Aplique `set_load` e `set_driving_cell` com valores típicos da biblioteca.

Parte 3 — Exploração dos limites do design

Após a síntese com as constraints iniciais, você irá variar o período do clock para encontrar a frequência máxima de operação do circuito e identificar o caminho crítico. Proceda da seguinte forma:

1. Parta do período inicial de 20 ns e reduza-o progressivamente em passos de 2 ns, re-executando a síntese a cada iteração.
2. Para cada período, registre na tabela abaixo o valor do slack reportado por `report_timing` e a área total reportada por `report_area`.
3. Identifique o menor período para o qual o slack ainda é não-negativo — esse é o período mínimo suportado pelo design com esta biblioteca e este nível de otimização.
4. Tente um período 2 ns abaixo do mínimo encontrado e observe como o Design Compiler reage: ele viola o timing ou encontra uma solução alternativa com maior área?
5. Retire a opção de síntese `-no-autoungroup` e veja se há alguma diferença nos reports de timing e área

Período	Frequência	Slack(ns)	Área	Timing

Parte 4 — Análise e discussão

No relatório, responda às seguintes perguntas com base nos resultados obtidos:

1. Qual é o caminho crítico do design? Quais módulos ele atravessa?
2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.
3. A partir de qual frequência o Design Compiler não conseguiu mais fechar o timing? O que acontece com o slack nesse ponto?
4. Considerando a FSM e o caminho de dados do projeto, qual estado ou transição você acredita estar no caminho crítico, e por quê?

5. Que modificação no RTL você proporia para reduzir o caminho crítico sem alterar o comportamento funcional do design?

10. Relatório Técnico

O relatório deve ser entregue em PDF (máximo 10 páginas) e conter:

1. Link para o repositório github do projeto
2. Diagrama de blocos
3. Diagrama de estados: redesenho manual ou com ferramenta, com todas as transições e saídas.
4. Descrição dos módulos: decisões de projeto e justificativas.
5. Waveforms anotadas: capturas do DVE identificando cada cenário de teste.
6. Saída do testbench: terminal mostrando os PASS/FAIL de cada verificação.
7. Resultados de síntese: área total, frequência máxima e slack do caminho crítico.
8. Análise: qual estado ou transição é o caminho crítico? Por quê?
9. Conclusão: dificuldades encontradas e lições aprendidas.

11. Estrutura de diretórios

```
grupo_NN_vending/
├── rtl/
│   ├── vending_pkg.sv
│   ├── credit_reg.sv
│   ├── memory.sv
│   ├── comparator.sv
│   ├── subtractor.sv
│   ├── control_unit.sv
│   └── vending_top.sv
├── sim/
│   └── tb_vending.sv
├── synth/
│   ├── synth.tcl
│   ├── vending.sdc
│   └── reports/
└── relatorio.pdf
```

#	Arquivo	Descrição
1	vending_pkg.sv	Package com parâmetros, encoding de estados e tipos
2	credit_reg.sv	Registrador de crédito acumulador (síncrono, com reset)
3	memory.sv	Memória síncrona 4×16 bits com leitura e escrita separadas
4	comparator.sv	Lógica combinacional: $can_sell = (credit \geq price) \ \&\& \ (stock > 0)$
5	subtractor.sv	Lógica combinacional: $change = credit - price$
6	control_unit.sv	FSM de Moore com 6 estados em SystemVerilog
7	vending_top.sv	Top-level: instancia e interconecta todos os módulos
8	tb_vending.sv	Testbench VCS com cenários de teste e verificação automática
9	synth.tcl	Script de síntese para Synopsys Design Compiler
10	vending.sdc	Constraints de timing (clock 50 MHz, I/O delays)

#	Arquivo	Descrição
11	relatorio.pdf	Relatório técnico com diagrama de estados, waveforms e resultados de síntese

12. Critérios de Avaliação

Item de Avaliação	Pontos	Critério
Módulos RTL compilando e corretos	20	Cada módulo compila sem erros com vcs e produz comportamento correto na simulação unitária
FSM de controle correta	20	Todos os 6 estados implementados; transições e saídas corretas para todos os cenários
Testbench self-checking	10	Cobre os 4 cenários obrigatórios; reporta PASS/FAIL automaticamente; possui timeout
Síntese com Design Compiler	10	Script executa sem erros; netlist gerada; relatório de área e timing disponível
Relatório técnico	40	Diagrama de estados desenhado, waveforms anotadas, análise do slack

Penalizações:

- -5 pontos por arquivo faltando na estrutura de diretórios
- -10 pontos se o top-level não compilar com vcs sem erros
- -5 pontos se o testbench não cobrir todos os 4 cenários obrigatórios
- -5 pontos se o script synth.tcl não executar até o fim sem erros fatais